case we contract over the `e` dimension, which is the node feature dimension, leaving us with two copies of the node dimension, which is why the output is of dimension $b \times n \times m$. When using Einsum, we can label the dimensions of each tensor with any alphabetic characters, but we must make sure that the dimension we wish to contract over is labeled with the same character for both tensors.

After the batch matrix multiplication, and we have the unnormalized adjacency matrix, we did `A = A / np.sqrt(self.node_size)` to rescale the matrix to reduce excessively large values and improve training performance; this is why we earlier referred to this is as *scaled* dot product attention.

In order to get the *Q*, *K*, and *V* matrices, as we discussed earlier, we took the output of the last convolutional layer which is a tensor of dimensions batch × channels × height × width, and we collapse the height and width dimensions into a single dimension of (height × width = n) for the number of nodes, since each pixel position will become a potential node or object in the node matrix. Thus we get an initial node matrix of N:$b \times c \times n$ that we reshape into N:$b \times n \times c$.

By collapsing the spatial dimensions into a single dimension, the spatial arrangement of the nodes is scrambled and the network would struggle to discover that certain nodes (which were originally nearby pixels) are related spatially. That is why we add two extra channel dimensions that encode the (x,y) position of each node before it was collapsed. We normalize the positions to be in the interval [0, 1], since normalization almost always helps with performance.

Adding these absolute spatial coordinates to the end of each node's feature vector helps maintain the spatial information, but it is not ideal since these coordinates are in reference to an external coordinate system, which means we're dampening some of the invariance to spatial transformations that a relational module should have, in theory. A more robust approach is to encode *relative* positions with respect to other nodes, which would maintain spatial invariance. However, this approach is more complicated, and we can